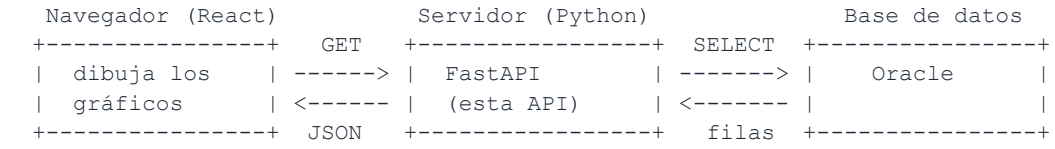
API de gráficos

Python + Oracle + React, paso a paso

Esta guía explica el proyecto **backend-python/** desde cero. Primero cuenta qué hace y los conceptos básicos. Después detalla los pasos para instalarlo, conectarlo a la base Oracle de la empresa, escribir los gráficos, usarlos desde React y desplegarlo en el servidor. Al final explica cada archivo línea por línea y da soluciones a los problemas más comunes.

1. Qué hace este proyecto

Es una **API de solo lectura**. Su única tarea es consultar tablas de Oracle y devolver los datos ya resumidos para que el front en React dibuje los gráficos.



Ejemplo concreto: React pide `GET /api/charts/ventas-por-mes`. La API ejecuta un `SELECT` que suma las ventas agrupadas por mes y devuelve:

```
[ { "mes": "2026-01", "total": 15000 },
  { "mes": "2026-02", "total": 23000 },
  { "mes": "2026-03", "total": 18500 } ]
```

React le pasa ese array directo a un gráfico de líneas, y listo.

Tres decisiones de diseño

- **No tiene login.** La app corre en un servidor de la empresa al que solo se accede por VPN, y solo lee datos. La VPN es la protección.
- **Solo lectura.** No hay endpoints para crear, modificar ni borrar. Además, el usuario de Oracle tiene que tener permiso solo de `SELECT`. Así, aunque hubiera un error en el código, la base no se puede modificar.
- **Los datos se agrupan en Oracle, no en Python.** Si una tabla tiene 2 millones de ventas, la API no las trae todas: Oracle hace el `SUM` / `GROUP BY` y devuelve 12 filas (una por mes). Es mucho más rápido y al navegador le llega muy poco.

2. Conceptos básicos

Concepto	Qué es
API	Un programa que responde pedidos HTTP con datos (en general JSON) en lugar de páginas web.
Endpoint	Cada URL que responde la API, por ejemplo `/api/charts/resumen`. En este proyecto, un endpoint = un gráfico.
JSON	El formato de texto de los datos: `{"mes": "2026-01", "total": 1500}`.
FastAPI	El framework de Python con el que está hecha la API (el equivalente a Express en Node).
uvicorn	El servidor que ejecuta la app de FastAPI y escucha en un puerto (8000).
python-oracledb	La librería oficial de Oracle para conectarse desde Python.
Pool de conexiones	Abrir una conexión a Oracle es lento. El pool deja algunas abiertas y las va prestando: cada pedido toma una, la usa y la devuelve.
Bind variables	La forma segura de pasar valores al SQL: se escribe `:desde` en la consulta y el valor va aparte. Así nadie puede "inyectar" SQL desde la URL.
.env	Archivo con la configuración y las contraseñas. No se sube a git.

Concepto	Qué es
Entorno virtual (.venv)	Carpeta con las librerías de este proyecto (como `node_modules`). Se activa antes de trabajar.
CORS	Regla del navegador: una página solo puede leer respuestas de otro origen (dominio o puerto distinto) si ese servidor lo autoriza. La API autoriza a la URL del front con `CORS_ORIGINS`.
Swagger (/docs)	Página que FastAPI genera sola. Lista los endpoints y permite probarlos desde el navegador.

Un poco más sobre CORS

Un **origen** es protocolo + dominio + puerto. En desarrollo, React corre en `http://localhost:5173` y la API en `http://localhost:8000`: cambia el puerto, así que son orígenes distintos. El navegador hace el pedido, pero **no le entrega la respuesta a React** salvo que la API diga "este origen está permitido". Si no lo dice, en la consola aparece `blocked by CORS policy`.

- En `CORS_ORIGINS` va la URL **del front**, no la de la API.
- Si el front y la API se sirven desde el mismo origen (por ejemplo `http://dashboard.empresa/` y `http://dashboard.empresa/api`), CORS no interviene.
- CORS no protege la API: lo aplica solo el navegador, y Postman o curl lo ignoran. Lo que protege la API es la VPN.

3. Paso a paso

Paso 1 · Verificar Python

Hace falta Python 3.9 o superior. En la terminal:

```
python3 --version        # tiene que decir 3.9 o más
```

En tu Mac está la 3.9.6, que alcanza. En el servidor de la empresa conviene 3.11 o 3.12.

Paso 2 · Crear y activar el entorno virtual

```
cd ~/Desktop/pyhtonSkeleton/backend-python
python3 -m venv .venv            # se hace UNA sola vez (ya está creado)
source .venv/bin/activate        # se hace CADA VEZ que abrís una terminal nueva
```

Al activarlo, el prompt de la terminal empieza con `(.venv)`. Desde ese momento `python` y `pip` usan las librerías del proyecto. Para salir: `deactivate`.

> **Ojo:** si al correr algo aparece `command not found: uvicorn` o `pytest`, casi siempre es porque te olvidaste de activar el entorno.

Paso 3 · Instalar las dependencias

```
pip install -r requirements-dev.txt
```

Instala las librerías de `requirements.txt` (fastapi, uvicorn, pydantic-settings, oracledb) más las de desarrollo (pytest, httpx). Es el `npm install` de Python.

Paso 4 · Pedirle los datos al DBA

Para conectarte a la base de la empresa necesitás:

Dato	Ejemplo	Para qué
Host (servidor)	`oracle01.empresa.local`	Dónde está la base.
Puerto	`1521`	Casi siempre es 1521.
SERVICE_NAME	`PRODPDB`	Nombre del servicio. Pedí el SERVICE_NAME, no el SID (son distintos).
Usuario y contraseña	`usr_dashboard`	Pedí un usuario con permiso **solo de SELECT**.
Esquema y tablas	`VENTAS.FACTURAS`	Qué tablas vas a leer y de qué esquema son.
Versión de Oracle	`19c`	12.1 o superior: no hay que instalar nada. Anterior: hace falta Instant Client (paso 5).

También verificá que el servidor donde va a correr la API pueda llegar a la base (que no lo bloquee un firewall). Si tenés SQL Developer o DBeaver, probá primero la conexión ahí con los mismos datos.

Paso 5 · Crear el archivo .env

```
cp .env.example .env
```

Abrí `.env` y completá:

```
ORACLE_USER=usr_dashboard
ORACLE_PASSWORD=la_clave_que_te_dio_el_DBA
ORACLE_DSN=oracle01.empresa.local:1521/PRODPDB
ORACLE_POOL_MAX=4
ORACLE_CLIENT_LIB_DIR=
CORS_ORIGINS=http://localhost:5173
```

- `ORACLE_DSN`: se arma como `host:puerto/SERVICE_NAME`.
- `ORACLE_POOL_MAX`: cuántas conexiones simultáneas como máximo. Para un dashboard interno, 4 sobra.
- `ORACLE_CLIENT_LIB_DIR`: dejalo vacío, salvo que la base sea anterior a 12.1. En ese caso, instalá *Oracle Instant Client* y poné acá la carpeta donde quedó.
- `CORS_ORIGINS`: la URL del front. En desarrollo con Vite es `http://localhost:5173`.

Ojo: el `.env` tiene la contraseña de la base: no lo subas a git (ya está en el `.gitignore`) ni lo mandes por chat o mail.

Paso 6 · Levantar la API

```
uvicorn app.main:app --reload
```

- `app.main:app` = "en el archivo `app/main.py`, usá la variable `app`".
- `--reload` reinicia la API cada vez que guardás un archivo. Solo para desarrollo.
- Si todo anda, ves `Uvicorn running on http://127.0.0.1:8000`.
- Si no se pudo conectar a Oracle, igual arranca, pero en el log aparece `No se pudo conectar a Oracle: ...` con el motivo (ver sección 5).

Paso 7 · Probar que funciona

1. Abrí `http://localhost:8000/health`. Si dice `{"status": "ok"}`, la conexión a Oracle anda. Si da error 503, revisá el log en la terminal.
2. Abrí `http://localhost:8000/docs`. Es Swagger: lista todos los endpoints.
3. Tocá un endpoint, después **Try it out** y **Execute**: muestra la respuesta real.

Mientras las consultas de ejemplo no se reemplacen por tablas reales, los endpoints de gráficos van a dar 503, porque la tabla `VENTAS` no existe. Es esperable: es el paso siguiente.

Paso 8 · Escribir los gráficos con las tablas reales

Los tres endpoints de `app/charts.py` son **ejemplos** sobre una tabla inventada `VENTAS(fecha, categoria, monto)`. Para cada gráfico que quieras:

1. **Pensá el gráfico:** ¿qué va en el eje X (o en las porciones de la torta) y qué número va en el eje Y? Por ejemplo: "cantidad de reclamos por zona" → X = zona, Y = cantidad.
2. **Escribí y probá el SELECT en SQL Developer/DBeaver** hasta que devuelva exactamente esas columnas, agrupado (`GROUP BY`) y ordenado.
3. **Copialo a un endpoint** en `app/charts.py`.
4. **Probalo en** `/docs`.

```
@router.get("/reclamos-por-zona", summary="Barras: reclamos por zona")
def reclamos_por_zona(desde: Optional[date] = None, hasta: Optional[date] = None):
    return db.fetch_all(
        f"""
        SELECT zona,
               COUNT(*) AS cantidad
        FROM soporte.reclamos
        WHERE {FILTRO_FECHAS}
        GROUP BY zona
        ORDER BY cantidad DESC
        """,
        {"desde": desde, "hasta": hasta},
    )
```

Qué hay que entender de este bloque:

- `@router.get("/reclamos-por-zona")`: registra la URL. Como el router tiene el prefijo `/api/charts`, queda en `/api/charts/reclamos-por-zona`.

- Los parámetros de la función (`desde`, `hasta`) se leen de la URL: `?desde=2026-01-01&hasta=2026-03-31`. Al declararlos como `date`, FastAPI valida que sean fechas y, si no lo son, responde 422 sin tocar la base. `Optional[...] = None` los hace opcionales.
- `FILTRO_FECHAS` es un pedazo de SQL reutilizable: si no mandás fechas, no filtra. Usa la columna `fecha`. Si tu tabla la llama distinto (por ejemplo `fecha_alta`), escribí el filtro a mano en esa consulta.
- Los nombres de las columnas del SELECT (`zona`, `cantidad`) son las claves del JSON que recibe React. Usá alias (`AS cantidad`) para que sean claros.
- Si el gráfico no necesita filtros, sacá los parámetros y el `WHERE`.

Ojo: los valores que vienen de la URL **siempre** como bind variables (`:desde` + diccionario). Nunca los metas en el string con f-string o concatenando: eso abre la puerta a la inyección SQL. El único f-string permitido es el de `{FILTRO_FECHAS}`, que es texto fijo nuestro.

Paso 9 · Correr los tests

```
pytest
```

Los tests no necesitan Oracle: reemplazan `db.fetch_all` por una función falsa que devuelve datos inventados. Verifican que los endpoints respondan bien, que una fecha inválida dé 422 y que un error de Oracle dé 503 sin mostrar el `ORA-xxxxx`. Cuando agregues gráficos, podés copiar `test_ventas_por_mes` como modelo.

Paso 10 · Mostrar los gráficos en React

En el proyecto de React, instalá una librería de gráficos. Recharts es simple y muy usada:

```
npm install recharts
```

Creá un archivo `.env` en el proyecto de React con la URL de la API:

```
VITE_API_URL=http://localhost:8000
```

Y un componente por gráfico:

```
import { useEffect, useState } from "react";
import { LineChart, Line, XAxis, YAxis, Tooltip, ResponsiveContainer } from "recharts";

const API = import.meta.env.VITE_API_URL;

export function VentasPorMes() {
  const [data, setData] = useState([]);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch(`${API}/api/charts/ventas-por-mes`)
      .then((res) => {
        if (!res.ok) throw new Error("No se pudieron cargar los datos");
        return res.json();
      })
      .then(setData)
      .catch((e) => setError(e.message));
  }, []);

  if (error) return <p>{error}</p>;

  return (
    <ResponsiveContainer width="100%" height={300}>
```

```
      <LineChart data={data}>
        <XAxis dataKey="mes" />      {/* columna del eje X */}
        <YAxis />
        <Tooltip />
        <Line dataKey="total" />     {/* columna del eje Y */}
      </LineChart>
    </ResponsiveContainer>
  );
}
```

La clave: `dataKey` tiene que coincidir con los nombres de las columnas del SELECT. Para barras se usa `BarChart` + `Bar`, y para torta `PieChart` + `Pie` (con `dataKey="total" nameKey="categoria"`).

Paso 11 · Desplegar en el servidor de la empresa

Hay dos formas. Elegí según lo que use el área de sistemas.

Opción A: Python directo en el servidor

```
# en el servidor, dentro de la carpeta del proyecto
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt          # sin las de desarrollo
cp .env.example .env                     # completar con datos de producción
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

- `--host 0.0.0.0`: escucha en todas las interfaces de red. Sin esto, solo responde desde el propio servidor.
- Sin `--reload`: en producción no hace falta.
- Para que arranque solo y se reinicie si se cae, conviene registrarlo como servicio del sistema (systemd en Linux, o un servicio de Windows). Eso normalmente lo resuelve el área de sistemas.

Opción B: Docker (si el servidor usa contenedores)

```
docker build -t api-graficos .
docker run -d --name api-graficos --env-file .env -p 8000:8000 --restart always api-graficos
```

En los dos casos:

- En `CORS_ORIGINS` agregá la URL real del front, por ejemplo `CORS_ORIGINS=http://dashboard.empresa.local`.
- En el React de producción, `VITE_API_URL` tiene que apuntar a la URL de la API en el servidor.
- Verificá `/health` desde una máquina conectada a la VPN.

4. Archivo por archivo

```
backend-python/
|-- app/
|   |-- __init__.py      vacío: marca la carpeta como paquete de Python
|   |-- main.py          crea la app: CORS, errores de Oracle, /health
|   |-- config.py        lee el .env
|   |-- db.py            pool de conexiones + fetch_all()
|   `-- charts.py        un endpoint por gráfico
|-- tests/
|   `-- test_charts.py   tests sin Oracle
|-- .env.example         plantilla de configuración (se copia a .env)
|-- .gitignore           qué no se sube a git
|-- requirements.txt     librerías para correr la app
```

```
|-- requirements-dev.txt + librerías para tests
|-- pytest.ini           dónde están los tests
|-- Dockerfile           para desplegar con Docker
`-- README.md            resumen de uso
```

app/config.py: la configuración

```python
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    ORACLE_USER: str = ""
    ORACLE_PASSWORD: str = ""
    ORACLE_DSN: str = "localhost:1521/SERVICIO"
    ORACLE_POOL_MAX: int = 4
    ORACLE_CLIENT_LIB_DIR: Optional[str] = None
    CORS_ORIGINS: str = "http://localhost:5173"

    @property
    def cors_origins_list(self) -> List[str]:
        return [o.strip() for o in self.CORS_ORIGINS.split(",") if o.strip()]

settings = Settings()
```

- `BaseSettings` (de pydantic-settings) completa cada atributo con la variable de entorno del mismo nombre, o con la del archivo `.env`. Si no existe ninguna, usa el valor de la derecha.
- Convierte tipos solo: en el `.env` todo es texto, pero `ORACLE_POOL_MAX` llega como número.
- `cors_origins_list` convierte `"http://a,http://b"` en la lista `["http://a", "http://b"]`.
- En el resto del código se usa así: `from app.config import settings` y después `settings.ORACLE_DSN`.

app/db.py: la conexión a Oracle

```python
_pool = None

def get_pool():
    global _pool
    if _pool is None:
        if settings.ORACLE_CLIENT_LIB_DIR:
            oracledb.init_oracle_client(lib_dir=settings.ORACLE_CLIENT_LIB_DIR)
        _pool = oracledb.create_pool(user=..., password=..., dsn=...,
                                     min=1, max=settings.ORACLE_POOL_MAX)
    return _pool
```

- `_pool` es una variable global. La primera vez que se llama a `get_pool()` se crea el pool; después devuelve siempre el mismo (patrón **Singleton**).
- Si hay `ORACLE_CLIENT_LIB_DIR`, activa el modo *thick* (con Instant Client) para bases viejas. Si no, usa el modo *thin*, en Python puro.
- `min=1, max=4`: deja al menos 1 conexión abierta y nunca más de 4.

```python
def fetch_all(sql, params=None):
    with get_pool().acquire() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})
        columns = [col[0].lower() for col in cur.description]
        return [dict(zip(columns, row)) for row in cur.fetchall()]
```

Es la única función que usan los endpoints. Línea por línea:

1. `get_pool().acquire()`: pide una conexión prestada al pool. `conn.cursor()`: crea un cursor, que es el objeto que ejecuta SQL.
2. El `with` garantiza que al terminar (aunque haya error) el cursor se cierre y la conexión vuelva al pool.
3. `cur.execute(sql, params)`: ejecuta la consulta pasando los valores de las bind variables por separado.
4. `cur.description` trae los nombres de las columnas (Oracle los da en MAYÚSCULA; se pasan a minúscula).
5. `cur.fetchall()` devuelve las filas como tuplas: `('2026-01', 15000)`. El `zip` las combina con los nombres: `{'mes': '2026-01', 'total': 15000}`. FastAPI convierte esa lista de diccionarios en JSON solo.

`close_pool()` cierra el pool cuando se apaga la app.

app/charts.py: los gráficos

```
router = APIRouter(prefix="/api/charts", tags=["Gráficos"])

FILTRO_FECHAS = "(:desde IS NULL OR fecha >= :desde) AND (:hasta IS NULL OR fecha < :hasta + 1)"
```

- `APIRouter` agrupa endpoints. `prefix` se antepone a todas las rutas del archivo. `tags` es el título con el que aparecen en `/docs`.
- `FILTRO_FECHAS` explicado: si `:desde` viene vacío (`IS NULL`), la condición es verdadera y no filtra; si viene una fecha, filtra `fecha >= desde`. Con `hasta` se suma 1 día (`:hasta + 1`) para incluir todo ese día, porque una columna DATE de Oracle también guarda la hora.

Después vienen los tres endpoints de ejemplo, todos con la misma forma (decorador, función con filtros opcionales, `db.fetch_all(SQL, params)`):

Endpoint	Devuelve	Gráfico sugerido
`/api/charts/ventas-por-mes`	`[{"mes", "total"}, ...]`	Líneas o barras
`/api/charts/ventas-por-categoria`	`[{"categoria", "total"}, ...]`	Torta o barras
`/api/charts/resumen`	`{"cantidad_ventas", "total_vendido", "ticket_promedio"}`	Tarjetas con números (KPIs)

`resumen` devuelve `rows[0]` (un objeto y no una lista) porque la consulta siempre da una sola fila. `NVL(x, 0)` devuelve 0 en lugar de `null` si no hay datos en el rango.

app/main.py: arma la aplicación

```
@asynccontextmanager
async def lifespan(app):
    try:
        db.get_pool()
    except oracledb.Error as exc:
        logger.error("No se pudo conectar a Oracle: %s", exc)
    yield
    db.close_pool()
```

`lifespan` es código que corre al **arrancar** (antes del `yield`) y al **apagar** (después). Al arrancar intenta crear el pool; si Oracle no responde, la app **no se cae**: lo deja anotado en el log y vuelve a intentar en el

próximo pedido. Al apagar cierra las conexiones.

```
app = FastAPI(title="API de gráficos", lifespan=lifespan)

app.add_middleware(CORSMiddleware,
                   allow_origins=settings.cors_origins_list,
                   allow_methods=["GET"], allow_headers=["*"])

app.include_router(charts.router)
```

- Crea la app. El `title` aparece en `/docs`.
- Configura CORS: solo los orígenes del `.env` y solo pedidos GET (la API no acepta nada más).
- `include_router` suma los endpoints de `charts.py`.

```
@app.exception_handler(oracledb.Error)
def oracle_error(request, exc):
    logger.exception("Error de Oracle en %s", request.url.path)
    return JSONResponse(status_code=503, content={"detail": "Error al consultar la base de datos"})

@app.get("/health")
def health():
    db.fetch_all("SELECT 1 FROM DUAL")
    return {"status": "ok"}
```

- **Manejador de errores:** si cualquier consulta falla, en vez de un error 500 con todo el detalle, responde 503 con un mensaje genérico. El error real (`ORA-00942: table or view does not exist`, etc.) queda en el log de la terminal, que es donde lo buscás. No se muestra en el navegador porque revelaría nombres de tablas y estructura interna.
- **/health:** hace la consulta más simple posible (`DUAL` es una tabla especial de Oracle de una fila). Si responde, la conexión anda; si no, el manejador de arriba devuelve 503.

tests/test_charts.py

```
def test_ventas_por_mes(monkeypatch):
    def fake_fetch_all(sql, params=None):
        return [{"mes": "2026-01", "total": 1500}]
    monkeypatch.setattr(db, "fetch_all", fake_fetch_all)

    res = client.get("/api/charts/ventas-por-mes", params={"desde": "2026-01-01"})
    assert res.status_code == 200
```

- `monkeypatch.setattr` reemplaza `db.fetch_all` por la función falsa solo durante ese test. Así se prueba la API sin necesitar Oracle.
- `client.get(...)` simula un pedido HTTP a la app.
- `assert` verifica el resultado. Si no se cumple, el test falla.
- Hay 4 tests: dos felices (ventas por mes, resumen) y dos no felices (fecha inválida → 422, error de Oracle → 503 sin `ORA-` en la respuesta).

Archivos de la raíz

Archivo	Para qué sirve
`requirements.txt`	Librerías para correr la app: fastapi, uvicorn, pydantic-settings, oracledb.
`requirements-dev.txt`	Las anteriores (`-r requirements.txt`) más pytest y httpx.
`.env.example`	Plantilla del `.env` con todas las variables y comentarios. Esta sí se sube a git; el `.env` no.
`.gitignore`	Evita subir `.venv/`, `.env` y archivos de caché.
`pytest.ini`	Le indica a pytest que los tests están en `tests/`.
`Dockerfile`	Receta para crear la imagen de Docker: Python 3.12, instala dependencias, copia `app/` y arranca uvicorn en el puerto 8000.
`README.md`	Resumen corto de uso.

5. Problemas comunes y cómo resolverlos

Los errores de Oracle aparecen en la terminal donde corre uvicorn. Buscá el código `ORA-` o `DPY-`.

Error	Qué significa	Qué hacer
`command not found: uvicorn`	El entorno virtual no está activado.	`source .venv/bin/activate`
`ORA-12541` / `DPY-6005`	No se llega al servidor (host o puerto mal, o firewall).	Revisar host y puerto en `ORACLE_DSN`. Verificar que estés en la VPN.
`ORA-12514`	El servidor existe, pero no conoce ese servicio.	El SERVICE_NAME está mal (¿te pasaron el SID?). Confirmarlo con el DBA.
`ORA-01017`	Usuario o contraseña incorrectos.	Revisar `ORACLE_USER` / `ORACLE_PASSWORD`.
`ORA-00942`	La tabla no existe o el usuario no tiene permiso para verla.	Escribir `ESQUEMA.TABLA` en el SQL. Pedirle al DBA el `GRANT SELECT`.
`ORA-00904`	Una columna del SELECT no existe.	Revisar el nombre de la columna en la tabla real.
`DPY-3010`	La base es demasiado vieja para el modo thin.	Instalar Oracle Instant Client y completar `ORACLE_CLIENT_LIB_DIR`.
`blocked by CORS policy` (consola del navegador)	La URL del front no está autorizada.	Agregarla a `CORS_ORIGINS` y reiniciar la API.
`422` en un endpoint	Un parámetro tiene formato inválido (por ejemplo, una fecha mal escrita).	Las fechas van como `AAAA-MM-DD`.
`address already in use`	El puerto 8000 está ocupado (quedó otra API corriendo).	Cerrar la otra terminal o usar `--port 8001`.